МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА - Российский технологический университет»

РТУ МИРЭА

ДИСЦИЛИНА **Системы искусственного интеллекта и большие данные**
(полное наименование дисциплины без сокращений)

ИНСТИТУТ **Искусственного интеллекта**

КАФЕДРА **Технологии искусственного интеллекта**
полное наименование кафедры

ВИД УЧЕБНОГО МАТЕРИАЛА **Практические занятия**
(в соответствии с пп.1-11)

ПРЕПОДАВАТЕЛЬ **Питинов Артем Вадимович**
(фамилия, имя, отчество)

СЕМЕСТР **3 семестр 2025**
(указать семестр обучения, учебный год)

Основы Python для обработки данных. Установка и настройка среды (Jupyter, Colab, VSCode). Загрузка данных из CSV/Excel. Источники экономических данных (макро: Росстат, ЦБ, МВФ; микро: CRM, транзакции, опросы, открытые данные). Загрузка данных Загрузка данных Росстата, ЦБ РФ, биржевых котировок. (CSV, Excel). Введение в NumPy. Индексация и срезы в многомерных массивах. Операции над данными в библиотеке Pandas. Базовые операции с DataFrame: выборка данных по условиям, добавление и удаление столбцов.Пример: расчет доходности портфеля активов; вычисление корреляционной матрицы для финансовых инструментов.

Ссылка на блокнот
https://colab.research.google.com/drive/1Pn5os6g4MMVEYwxpFpAGxlXa6Q-oZmqY?usp=sharing
0. Установка и запуск среды

 Выберите любой из трёх вариантов.

a) Google Colab

1. Зайдите на https://colab.research.google.com и создайте новый notebook.
2. У Colab уже предустановлены pandas, numpy, openpyxl (для Excel).

b) Jupyter Notebook (Anaconda)

1. Скачайте и установите Anaconda Distribution.
2. Запустите Anaconda Navigator → Jupyter Notebook.
3. Создайте notebook и, при необходимости, установите недостающие пакеты:

```
conda install pandas openpyxl xlrd
```

c) VS Code + Jupyter extension

1. Установите VS Code и расширение *Jupyter*.
2. Откройте .ipynb-файл — появится интерактивный интерфейс.
3. Через терминал выполните:

```
pip install pandas openpyxl xlrd
```

Где можно брать данные для заданий?

* Соревнования Kaggle](https://www.kaggle.com/datasets)
* Google Datasets](https://datasetsearch.research.google.com/)
* Сайт huggingface](https://huggingface.co/datasets)

* □ Федеральная служба государственной статистики](https://rosstat.gov.ru/folder/10705)
* □ ЕМИСС Государственная статистика](https://fedstat.ru/)

Инструменты

Будем использовать библиотеки:

* [[doc] □ NumPy](https://numpy.org/) — поддержка больших многомерных массивов и быстрых математических функций для операций с этими массивами.
* [[doc] □ Scikit-learn](https://scikit-learn.org/stable/) — ML алгоритмы.
* [[doc] □ Pandas](https://pandas.pydata.org/) — удобная работа с табличными данными.
* [[doc] □ PyTorch](https://pytorch.org/) — фреймворк машинного обучения.
* [[doc] □ Matplotlib](https://matplotlib.org/) — библиотека для визуализации. Вывод различных графиков.
* [[doc] □ Seaborn](https://seaborn.pydata.org/) — еще одна библиотека для визуализации .

```
#Если работаете в локальном Jupyter/VS Code и модулей нет — раскомментируйте
#!pip install pandas openpyxl xlrd -q
import pandas as pd
```

1. Загрузка тренировочного набора данных
Обзор доступных данных

Датасет: IBM HR Analytics Employee Attrition & Performance Описание: Данные о сотрудниках компании IBM, включая признаки увольнения (Attrition), удовлетворенности, условий труда, зарплаты и др. Актуальность для инноватики: высокая текучесть кадров снижает эффективность инновационных проектов.

```
# Загрузка архива с Google Drive
!gdown 1ssiVlJiAYRfkpnVtQwg-UE253RvrrdDx -O IBM_HR_dataset.zip

# Распаковка
!unzip IBM_HR_dataset.zip

# Загрузка данных в DataFrame
df = pd.read_csv('WA_Fn-UseC_-HR-Employee-Attrition.csv')  # Убедитесь, что имя файла верное
print("Датасет загружен.")
df
```

```
Датасет загружен.
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	Emplo
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life Sciences	1	
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life Sciences	1	
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	Other	1	
3	33	No	Travel_Frequently	1392	Research & Development	3	4	Life Sciences	1	
4	27	No	Travel_Rarely	591	Research & Development	2	1	Medical	1	

2. Первичный обзор структуры данных

Методы, которые мы изучим:

• shape — размеры ((строки, столбцы))

• columns — список имён столбцов

• head(n) / tail(n) — первые / последние n строк

• sample(n) — случайные n строк

• info() — типы данных, пропуски, память

• describe() — базовые статистики столбцов

```
print("Размер датасета:", df.shape)

print(" Заголовки столбцов:")
print(df.columns.tolist())

 print("Первые 3 строки:")
df.head(3)
```

```
Первые 3 строки:
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	Education	EducationField	EmployeeCount	Empl
0	41	Yes	Travel_Rarely	1102	Sales	1	2	Life Sciences	1	
1	49	No	Travel_Frequently	279	Research & Development	8	1	Life Sciences	1	
2	37	Yes	Travel_Rarely	1373	Research & Development	2	2	Other	1	

3 rows × 35 columns

```
 print("Сводная информация:")
df.info()
```

```
Сводная информация:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Age                       1470 non-null   int64
 1   Attrition                 1470 non-null   object
 2   BusinessTravel            1470 non-null   object
 3   DailyRate                 1470 non-null   int64
 4   Department                1470 non-null   object
 5   DistanceFromHome          1470 non-null   int64
 6   Education                 1470 non-null   int64
 7   EducationField            1470 non-null   object
 8   EmployeeCount             1470 non-null   int64
 9   EmployeeNumber            1470 non-null   int64
 10  EnvironmentSatisfaction   1470 non-null   int64
 11  Gender                    1470 non-null   object
 12  HourlyRate                1470 non-null   int64
 13  JobInvolvement            1470 non-null   int64
 14  JobLevel                  1470 non-null   int64
 15  JobRole                   1470 non-null   object
 16  JobSatisfaction           1470 non-null   int64
 17  MaritalStatus             1470 non-null   object
 18  MonthlyIncome             1470 non-null   int64
 19  MonthlyRate               1470 non-null   int64
 20  NumCompaniesWorked        1470 non-null   int64
 21  Over18                    1470 non-null   object
 22  OverTime                  1470 non-null   object
 23  PercentSalaryHike         1470 non-null   int64
 24  PerformanceRating         1470 non-null   int64
 25  RelationshipSatisfaction  1470 non-null   int64
 26  StandardHours             1470 non-null   int64
 27  StockOptionLevel          1470 non-null   int64
 28  TotalWorkingYears         1470 non-null   int64
 29  TrainingTimesLastYear     1470 non-null   int64
 30  WorkLifeBalance           1470 non-null   int64
 31  YearsAtCompany            1470 non-null   int64
 32  YearsInCurrentRole        1470 non-null   int64
 33  YearsSinceLastPromotion   1470 non-null   int64
 34  YearsWithCurrManager      1470 non-null   int64
dtypes: int64(26), object(9)
memory usage: 402.1+ KB
```

Выведем пять последних строк (tail).

df.tail()

#Выведем 3 случайных столбца из этой выборки (sample).

df.sample(3)

Введение в NumPy
NumPy представляет собой библиотеку нацеленную на работу с многомерными массивами и матричными структурами данных. Также она может быть задействована для выполнения различных математических операций с массивами.

```
import numpy as np

mass1 = np.array([1, 2, 3], int)
mass2 = np.array([4, 5, 6], int)
print(mass1[1])

print(mass1 + mass2)
```

```
print(mass1 - mass2)

print(mass1 * mass2)

print(mass1 / mass2)

print(mass1 % mass2)

print(mass1 ** mass2)

# Пример инициализации двумерного массива:
mass3 = np.array([[1, 2], [3, 4]], float)
mass4 = np.array([[5, 6], [7, 8]], float)

print(mass3 * mass4)

mass5 = np.array([[1, 2], [3, 4], [5, 6]], float)
mass6 = np.array([-1, 3], float)
print(mass5 * mass6)
```

Базовые операции над массивами

```
mass7 = np.array([2, 4, 5], float)
print ('СУмма = ' + str(mass7.sum()))

print ('Результат перемножения = ' + str(mass7.prod()))

print ('Среднее значение арифметическое = ' + str(mass7.mean()))

print ('Минимальное = ' + str(mass7.min()))

print ('Индекс Минимального = ' + str(mass7.argmin()))

mass8 = np.array([7, 4, 5, 1], float)
print (sorted(mass8))

mass9 = np.array([[1, 2], [3, 4], [5, 6]], float)
for x in mass9:
  print(x)

df.apply(np.max)
```

```
df.apply(np.max)
```

	0
Age	60
Attrition	Yes
BusinessTravel	Travel_Rarely
DailyRate	1499
Department	Sales
DistanceFromHome	29
Education	5
EducationField	Technical Degree
EmployeeCount	1
EmployeeNumber	2068
EnvironmentSatisfaction	4
Gender	Male

3. Выборка строк и столбцов в датафрейме

Базовые способы:

1. **По имени столбца** - df['col'] (например, df['housing_median_age'])
2. **Срез строк по индексу** - df[5:10] (это строки 5-9)
3. **loc (метка)** - df.loc[строки, столбцы] (например, df.loc[0:4, 'housing_median_age'])
4. **iloc (позиция)** - df.iloc[строки, столбцы] (например, df.iloc[:5, :3])

#Один столбец

ages = df.loc[5:10,'Age']
ages

# Выбор нескольких столбцов
subset = df[['Age', 'MonthlyIncome', 'JobSatisfaction', 'Attrition']]
subset.head(10)

Добавим бинарный столбец: высокая зарплата (выше среднего)

df['HighIncome'] = (df['MonthlyIncome'] > df['MonthlyIncome'].mean())

df[['MonthlyIncome', 'HighIncome']].head(10)

df[['Оценка риелтора', 'Высоко оцененные']].head(10)

Добавление и удаление столбцов

```
# Добавление нового случайного столбца
np.random.seed(42)  # для воспроизводимости
df['ProjectCount'] = np.random.randint(1, 10, size=len(df))
df.head(3)
```

```
df_new = df['ProjectCount']
df_new
```

```
df.rename(columns = {'ProjectCount':'Jobs done'}, inplace=True)
df
```

```
df = df.drop(columns=['Jobs done'])
df
```

Получение строк, удовлетворяющих определенному условию

```
# Фильтрация: сотрудники с высоким уровнем стресса (долго работают)
print("Сотрудники, работающие сверхурочно:")
overtime_workers = df[df['OverTime'] == 'Yes']
overtime_workers[['EmployeeNumber', 'Age', 'Department', 'MonthlyIncome']].head()
```

```
# Уволившиеся сотрудники
print("Уволившиеся (Attrition = 'Yes'):")
attrited = df[df['Attrition'] == 'Yes']
attrited[['Age', 'JobRole', 'MonthlyIncome', 'YearsAtCompany']].head()
```

```
print(df[df['Education'] > 2])
```

4. Сохранение результата

Мы хотим выгрузить обработанный датафрейм в:

• CSV (to_csv)

• Excel (to_excel)

#Без индекса в CSV

```
# Сохранение в CSV без индекса
df.to_csv('hr_processed.csv', index=False)
print("Сохранено как 'hr_processed.csv'")

# Сохранение в Excel
df.to_excel('hr_processed.xlsx', index=False)
print("Сохранено как 'hr_processed.xlsx'")
```

Задание

1.Используя данный набор данных создайте новый, состоящий только из тех строк, где значение в столбце Целевая.Цена выше 800000. Сохраните его в отдельный csv файл.
2.Откройте данный файл и добавте в него новый столбец в котором значения столбца Оценка риелтора увеличены на 2

Самостоятельное задание

Расчет доходности портфеля активов; вычисление корреляционной матрицы для финансовых инструментов.

Изучить представленный ниже набор данных о катировка разных компаний с периодичностью в 1 день, Выведя о нем справочную информацию.

```
# установка библиотек
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. Исходные параметры

# Установка yfinance при необходимости
```

```
!pip install yfinance -q

import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Параметры
TICKERS = ['IBM', 'MSFT', 'GOOGL', 'AAPL', 'TSLA']
START_DATE = '2024-01-01'
END_DATE = '2024-12-31'
INTERVAL = '1d'
WEIGHTS = np.array([1/5] * 5)  # равные доли

# Загрузка цен закрытия
prices = yf.download(
    tickers=TICKERS,
    start=START_DATE,
    end=END_DATE,
    interval=INTERVAL
)['Close']

print("Первые 15 строк цен:")
prices.head(15)
print("\nПоследние 8 строк:")
prices.tail(8)
print("\nИнформация о датасете:")
prices.info()
```

Описание

Функция yf.download: Эта функция используется для загрузки исторических данных по указанным тикерам (акциям, индексам и т.д.). В данном случае:

– tickers=TICKERS: переменная TICKERS содержит список тикеров, для которых нужно загрузить данные (например, ['AAPL', 'GOOGL']).

– start=START_DATE: переменная START_DATE указывает начальную дату для загрузки данных.

– end=END_DATE: переменная END_DATE указывает конечную дату для загрузки данных.

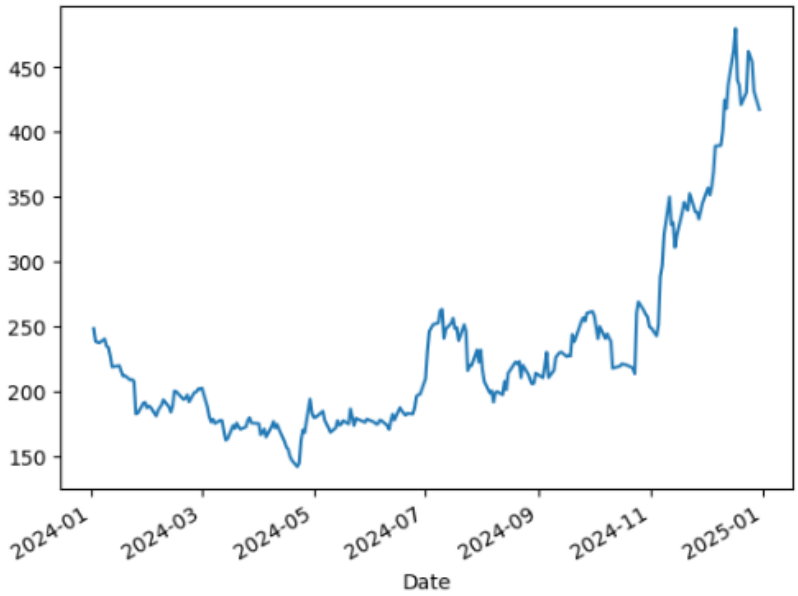
– interval=INTERVAL: переменная INTERVAL определяет интервал между загружаемыми данными (например, '1d' для дневных данных).

['Close']: После загрузки данных, код берёт только столбец с ценами закрытия (Close). Это означает, что в результате будут доступны только цены на конец дня для каждого тикера.

Переменная prices: Результат выполнения функции yf.download будет сохранён в переменной prices. Это будет DataFrame (таблица) с историческими ценами закрытия для указанных тикеров за заданный период.

1. Выведите первые 15 строк набора
2. Выведите последние 8 строк набора
3. Отобразите справочную информацию о самом наборе

```
# Посмотрим как, например, менялись в течении года цены на акции тесла
prices.TSLA.plot()
```



3.Доходность за день

returns = prices.pct_change().dropna()

print("Дневная доходность (первые 5 строк):")

returns.head()

Выведите полученный результат

Доходность портфеля

port_ret_series = (returns * WEIGHTS).sum(axis=1)

print("Ежедневная доходность портфеля (первые 5 дней):")

port_ret_series.head()

выведите полученный результат

5. Корреляционная матрица

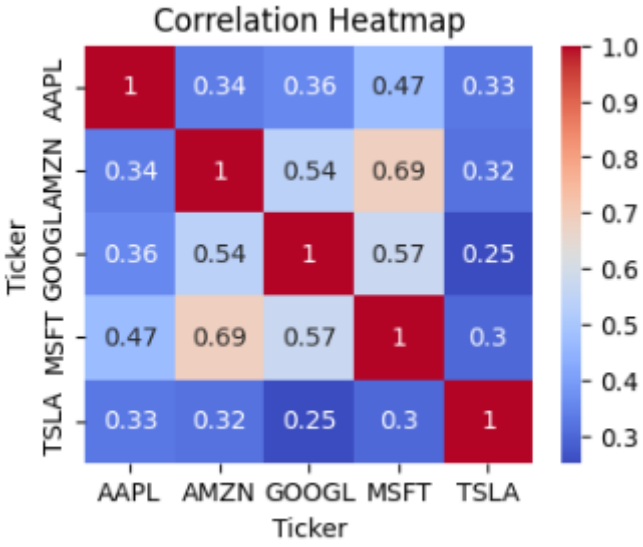
# посмотрим, взаимосвязаны ли между собой выбранные нами акции
corr_matrix = returns.corr().round(3)
print('\nКорреляционная матрица доходностей:')
corr_matrix

Корреляционная матрица доходностей:

Ticker	AAPL	GOOGL	IBM	MSFT	TSLA
Ticker					
AAPL	1.000	0.359	0.119	0.470	0.328
GOOGL	0.359	1.000	0.100	0.572	0.251
IBM	0.119	0.100	1.000	0.292	-0.028
MSFT	0.470	0.572	0.292	1.000	0.300
TSLA	0.328	0.251	-0.028	0.300	1.000

нарисуем тепловую карту - просто для красоты

```
import seaborn as sns
plt.figure(figsize=(4, 3))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```



Например, IBM наиболее связана с Microsoft

Задание

1.Нарисовать графики остальных акций
2.Вопрос посложнее - как посчитать доходность портфеля за год?
3.Добавьте в потрфель золото 'GLD' и поменяйте все доли на 1/6